STATE-SPACE SEARCH

Any-Path Search

Cost and Performance Properties

n.h.reyes@massey.ac.nz

Reference: **MIT OpenCourseWare**

Worst case scenarios

- Worst case running time
- Worst case space

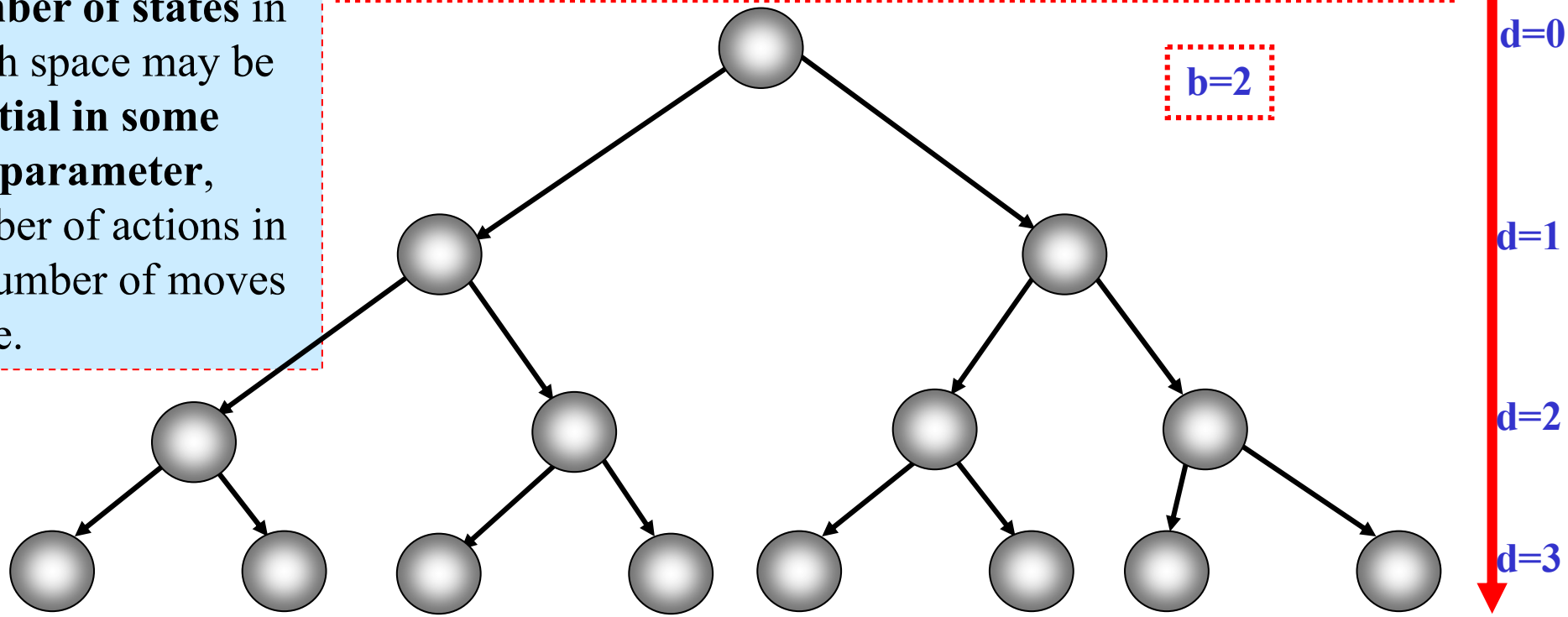
back

SEARCH

Worst Case Running Time for all Searches

The **number of states** in the search space may be **exponential in some** "**depth**" **parameter**, e.g. number of actions in a plan, number of moves in a game.

Max Time is proportional to the **Max. number of Nodes visited**.



Tree represents the search space
d is depth
b is branching factor
Number of States in the tree = $b^d < (b^{d+1} - 1)/(b-1) < b^{d+1}$

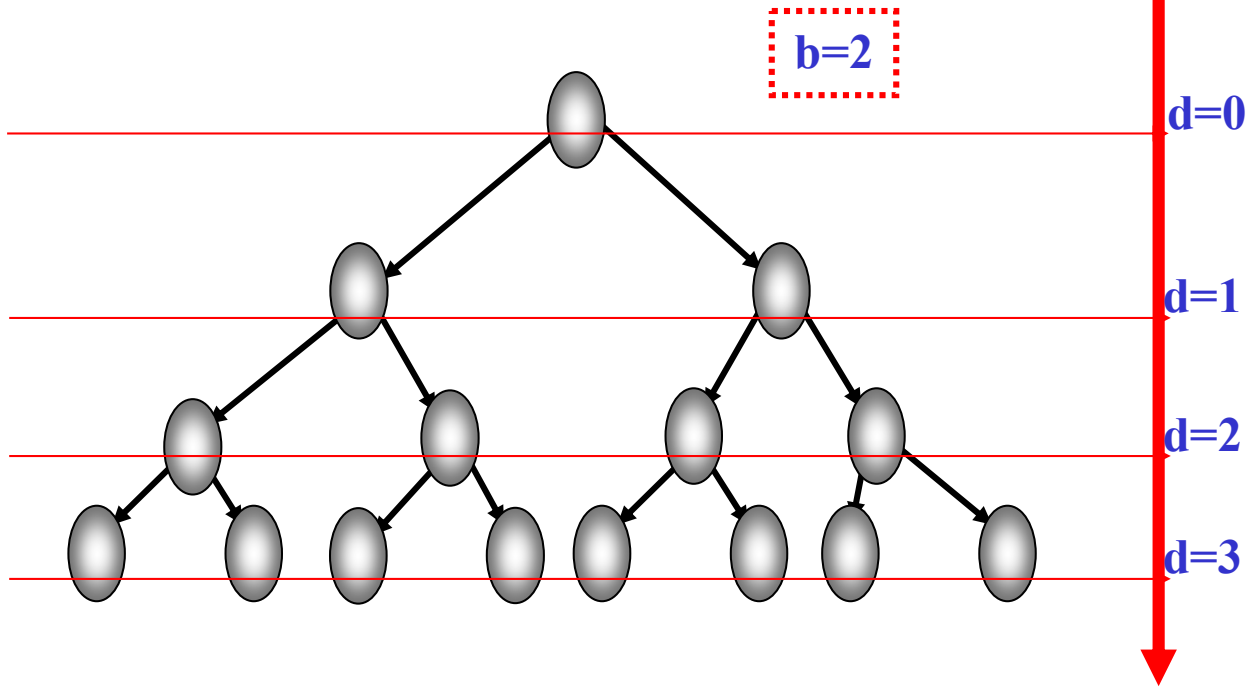
ck

Worst Case Running Time for all Searches

In the worst case, all the searches, **with or without visited list** may have to visit each state at least once.

So, all searches will have worst case running times that are at least proportional to the total number of states and therefore exponential in the "**depth**" parameter.

Max Time is proportional to the **Max. number of Nodes visited**.

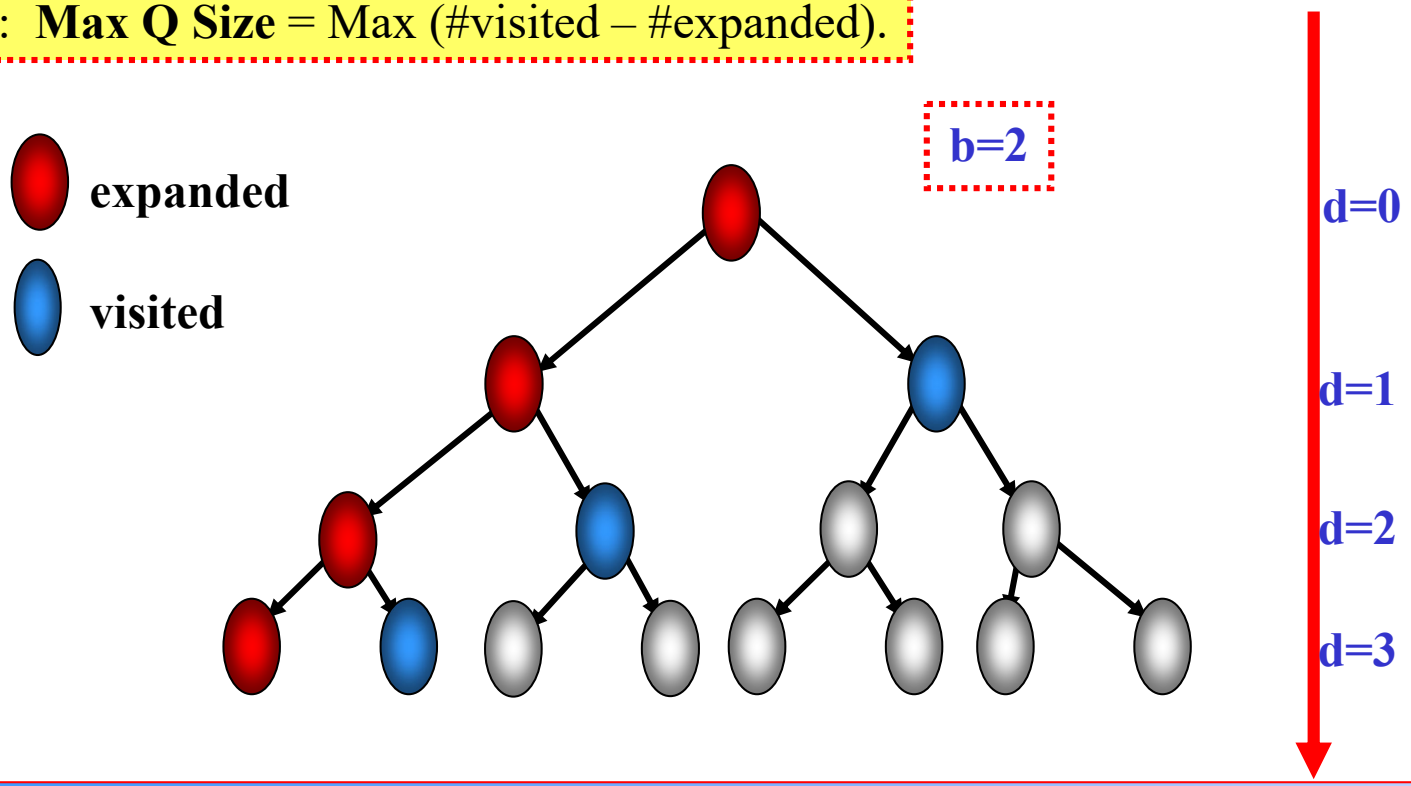


d is depth
b is branching factor
Number of States in the tree = $b^d < (b^{d+1} - 1)/(b-1) < b^{d+1}$

back

Worst Case Space for Depth-First Search

Dominant Factor: **Max Q Size** = Max (#visited – #expanded).



Depth-first Search: maximum Q size (always less than bd): $(b-1)d \approx bd$

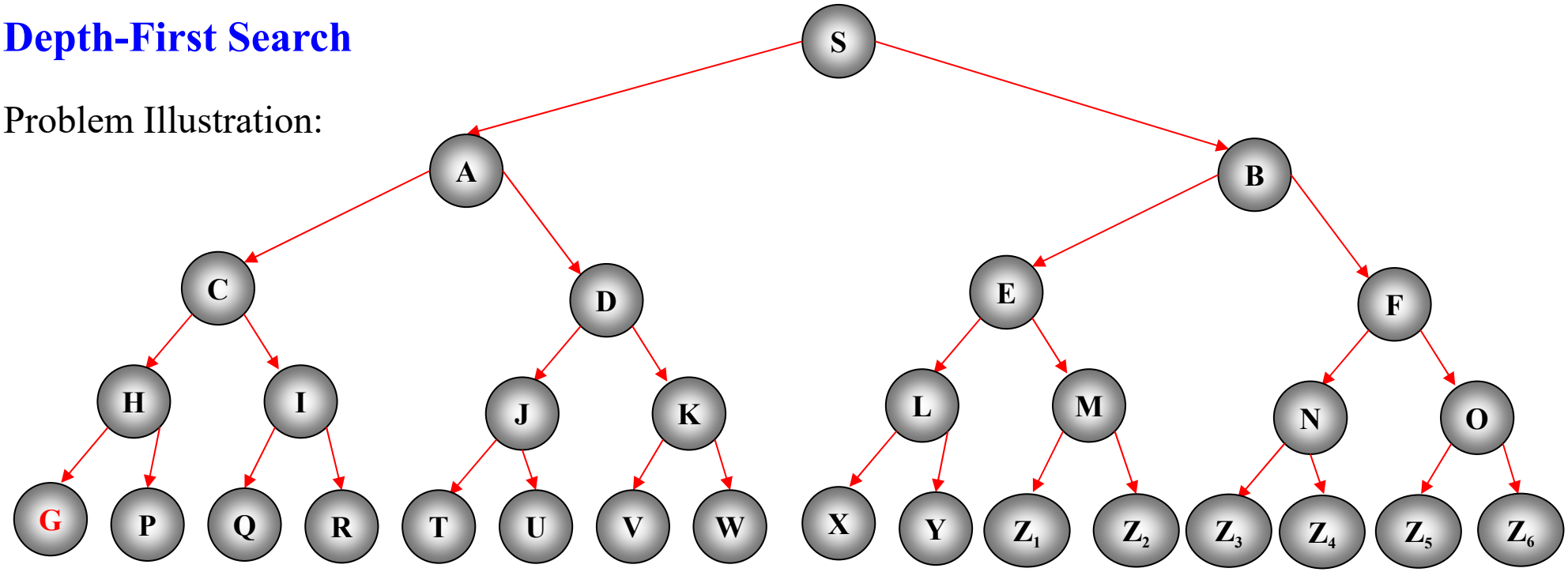
The **Q** holds the number of unexpanded "siblings" of the nodes along the path that we are currently considering.
The **space requirements** are **linear** in **d**.

Exercise: Worst Case Space for Depth-First Search

Dominant Factor: **Max Q Size** = Max (#visited – #expanded).

Depth-First Search

Problem Illustration:



In search for a path to **G**, what is the max Q length incurred, and when does it happen (at which depth)?

Exercise: Worst Case Space for Depth-First Search

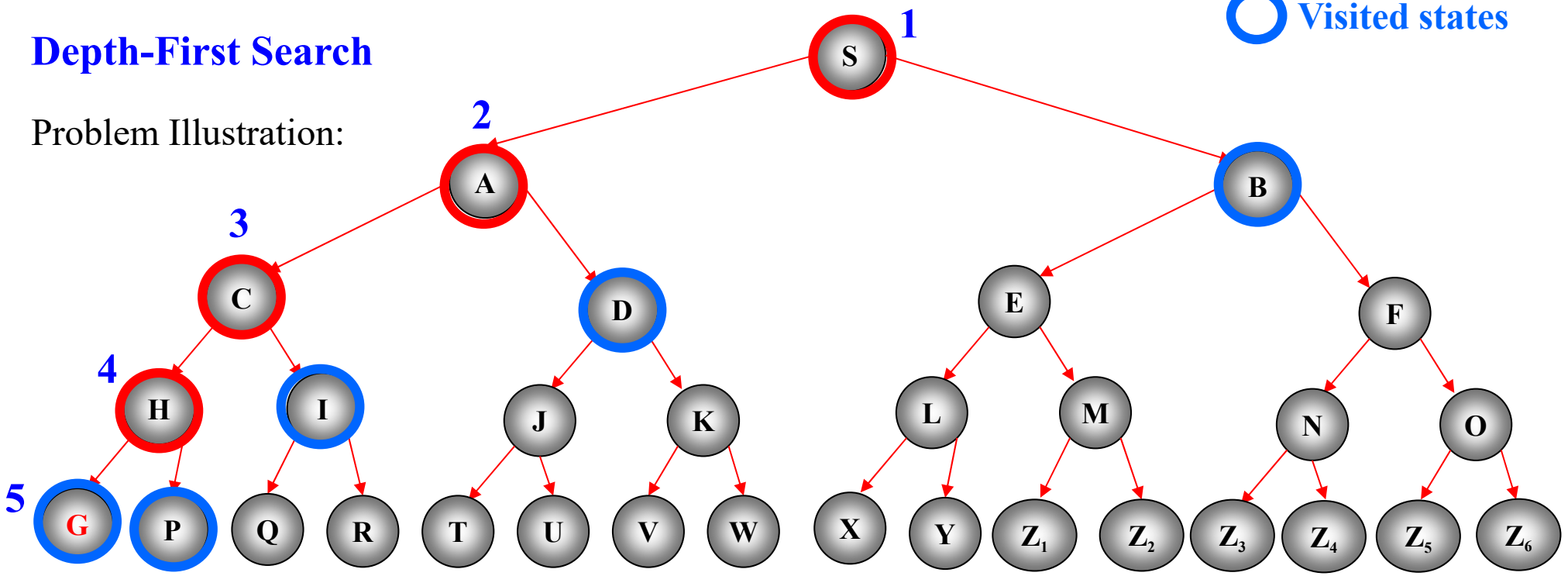
Dominant Factor: **Max Q Size** = Max (#visited – #expanded).

Expanded states

Visited states

Depth-First Search

Problem Illustration:

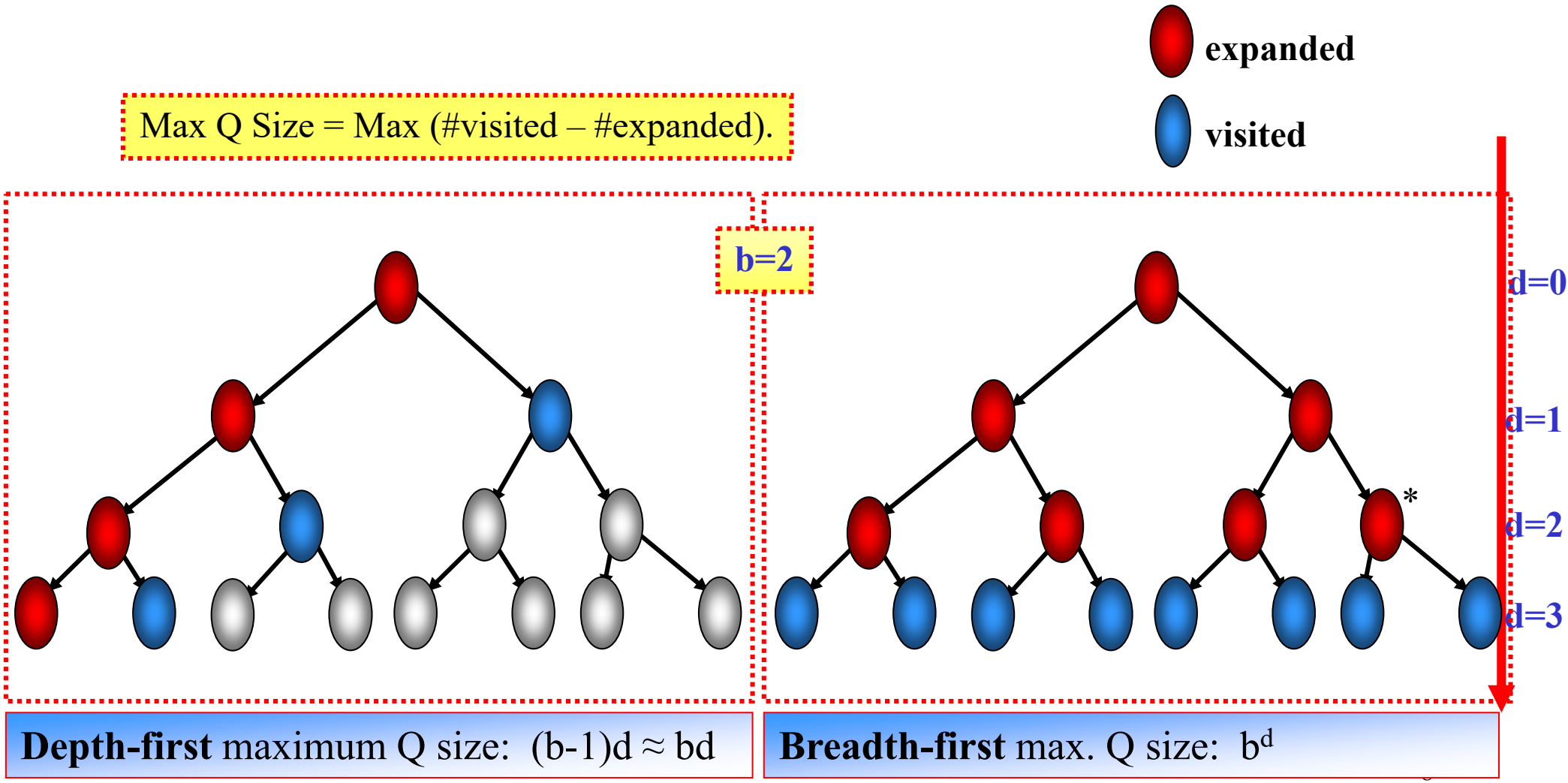


In search for a path to G, what is the max Q length incurred, and when does it happen (at which depth)?

Max Q Length = Max(#visited - #expanded) = 9 – 4 = 5

SEARCH

Worst Case Space (DFS vs. BFS)



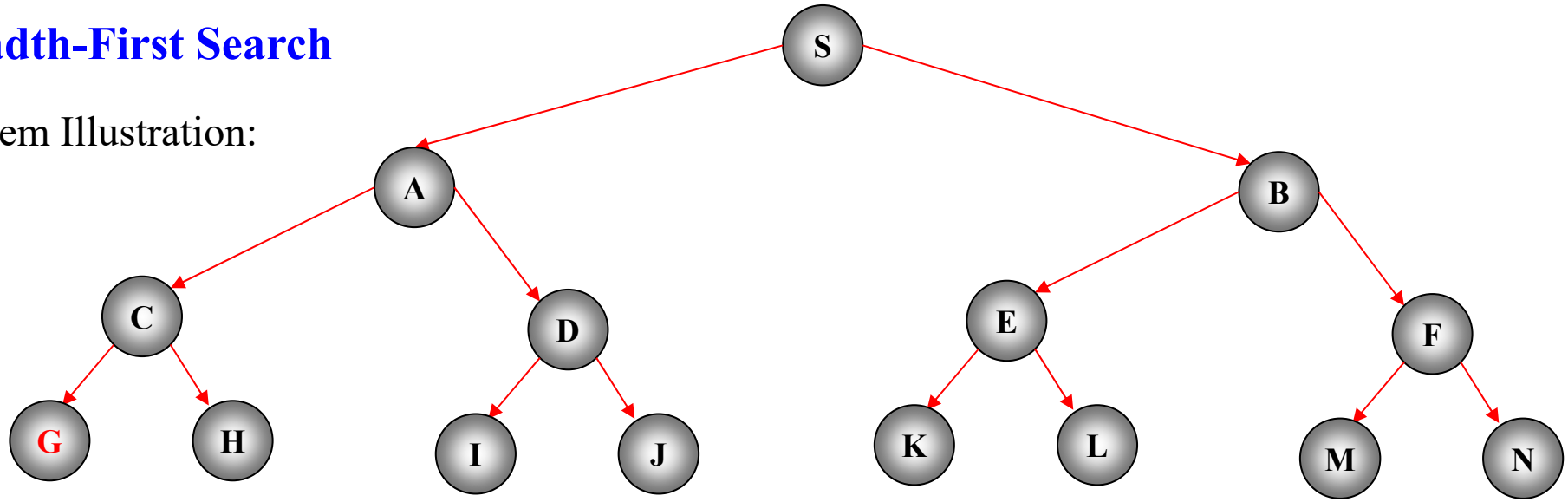
back

Exercise: Worst Case Space for Breadth-First Search

Dominant Factor: **Max Q Size** = Max (#visited – #expanded).

Breadth-First Search

Problem Illustration:



In search for a path to **G**, what is the max Q length incurred, and when does it happen (at which depth)?

Exercise: Worst Case Space for Breadth-First Search

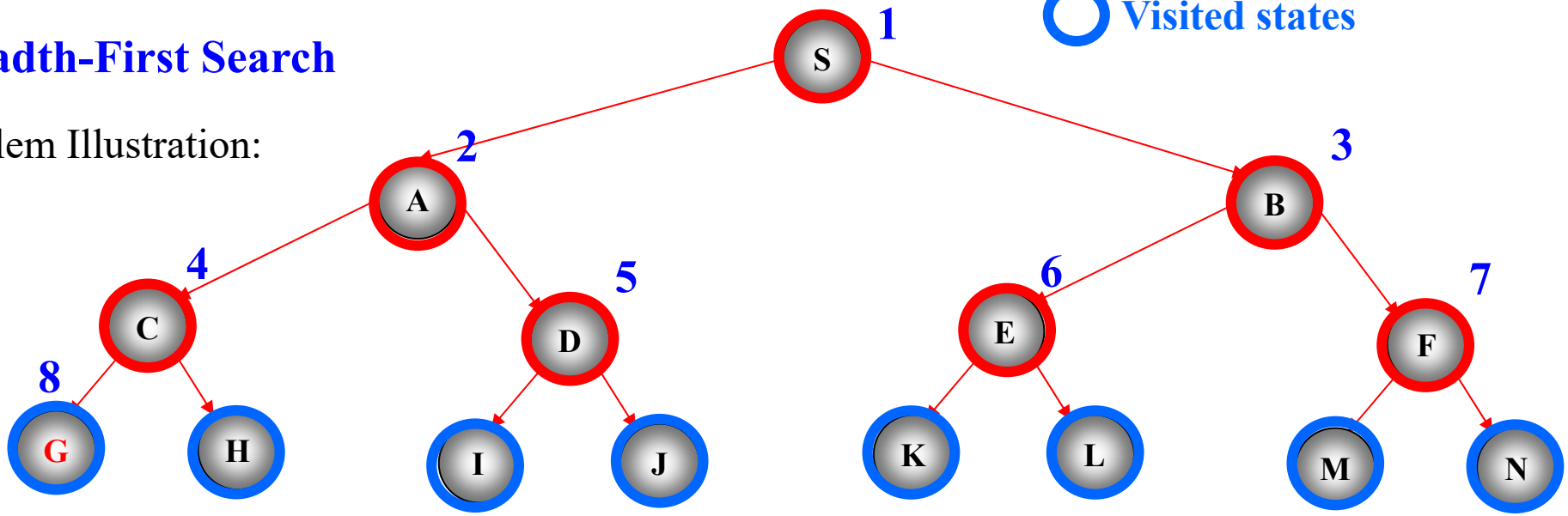
Dominant Factor: **Max Q Size** = Max (#visited – #expanded).

Expanded states

Visited states

Breadth-First Search

Problem Illustration:



In search for a path to **G**, what is the max Q length incurred, and when does it happen (at which depth)?

Max Q Length = Max(#visited - #expanded) = 15 – 7 = 8

SEARCH

Cost and Performance of Any-Path Methods

Searching a tree with branching factor **b** and depth **d** (**without using a Visited List**)

Search Method	Worst Time	Worst Space	Fewest states?	Guaranteed to find a path?
Depth-First	b^{d+1}	bd	No	Yes*
Breadth-First	b^{d+1}	b^d	Yes	Yes
Best-First	b^{d+1} **	b^d	No	Yes*

*If there are no indefinitely long paths in the search space
**Best-First needs more time to locate the best node in Q.

Worst case time is proportional to the number of nodes added to Q.
Worst case space is proportional to maximal length of Q.

Considering that all states are unique

Cost and Performance of Any-Path Methods

Searching a tree with branching factor **b** and depth **d** (**with Visited List**)

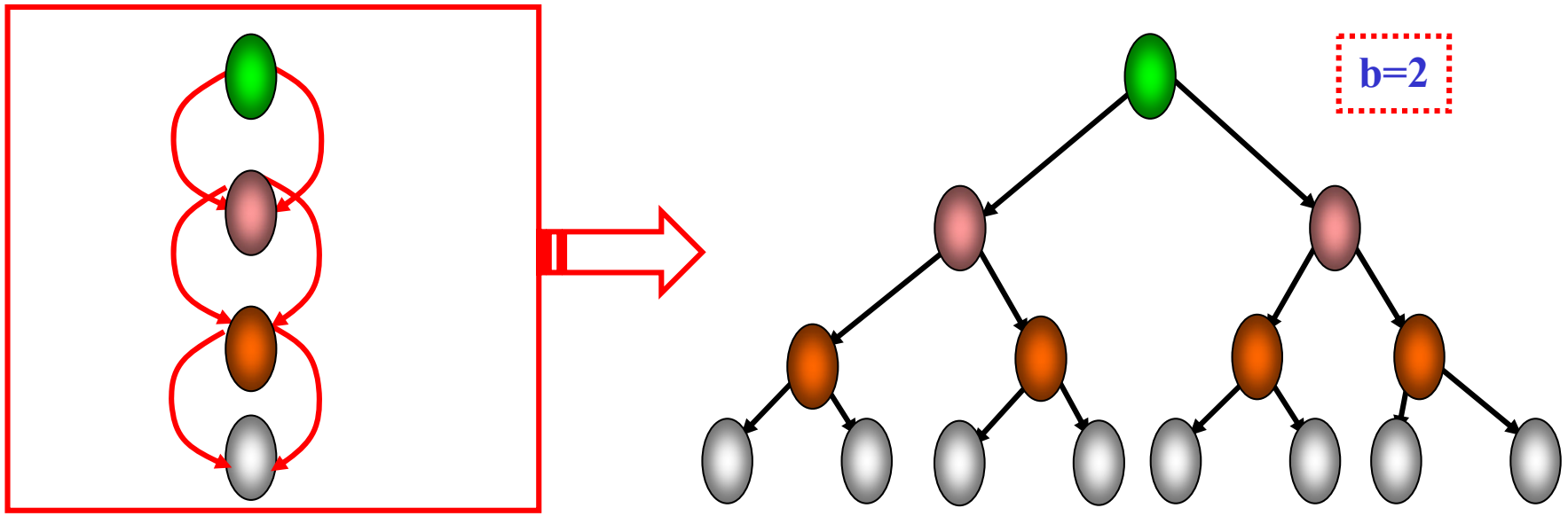
Search Method	Worst Time	Worst Space	Fewest states?	Guaranteed to find a path?
Depth-First	b^{d+1}	~~bd~~ b^{d+1}	No	Yes*
Breadth-First	b^{d+1}	~~b^d~~ b^{d+1}	Yes	Yes
Best-First	b^{d+1} **	~~b^d~~ b^{d+1}	No	Yes*

*If there are no indefinitely long paths in the search space
**Best-First needs more time to locate the best node in Q.

Worst case time is proportional to the number of nodes added to Q.
Worst case space is proportional to maximal length of Q **and Visited list**.

States vs. Path

Using the Visited List is a way of spending space to limit the worst case running time.



Using a Visited list, the **worst-case time performance** is limited by the **number of states** in the search space **rather than the number of paths** through the nodes in the space (which may be exponentially larger than the number of states).

Using a Visited list helps prevent loops; that is, no path visits a state more than once. However, using the Visited list for very large spaces may not be appropriate as the space requirements would be prohibitive.

back